

Observe, Expand, Recover: An Observability-Guided Agent for Safe Query Expansion

An Honest Account of What Runtime Telemetry Can and Cannot Buy a Query Expansion Agent

Author One*, Author Two†

*Affiliation One, email@example.com

†Affiliation Two (AI Observability), email2@example.com

Abstract—Large language model (LLM) query expansion is typically applied either unconditionally or not at all, despite well-documented evidence that unconditional expansion is net-harmful on a non-trivial fraction of queries. We formulate query expansion instead as a bounded, observable agentic decision, and evaluate it under a train/validation/test discipline strict enough to survive review: every retrieval-config choice, verifier-model selection, and recovery threshold is fixed on a held-out TripClick TAIL *validation* split before a single frozen touch of the *test* split, per research question. We study three questions: (RQ1) can qrel-free telemetry predict whether an expansion will help or hurt; (RQ2) does a closed-loop agent conditioned on that telemetry outperform an otherwise identical static, failure-conditioned expansion pipeline; and (RQ3) can observability-driven recovery detect and correct deliberately injected failures. A first pass at this study used an under-tuned BM25 baseline (nDCG@10 0.228, well below the 0.267 literature reference for this split); correcting it — stemming plus title-field weighting, selected entirely on validation — raises BM25 to nDCG@10 0.315, *above* the reference point, and changes every downstream conclusion. On the frozen test set, the selected verifier (nothing beat a plain hand-picked baseline on validation, so we deploy the honestly-fit model anyway) generalizes better than its own noisy validation estimate suggested: Pearson $r = 0.114$ ($p < 0.001$), ROC-AUC 0.552, against click-derived relevance labels never seen during selection. Neither static expansion nor the closed-loop agent significantly outperforms corrected BM25 (static: +0.0007, agent: +0.0009 nDCG@10, both 95% CI crossing zero), nor does the agent beat static (+0.0002). Critically, this is not because no headroom exists: a retrospective oracle with perfect per-query accept/reject knowledge would gain +0.0246 nDCG@10 over BM25 ($p < 0.001$) — roughly 30× what the deployed verifier actually captures — showing the failure mode is a weak *detector*, not an absent *opportunity*. Fault-injection recovery is fault-type-dependent: a content-level entity-injection fault is not significantly aided (mean nDCG@10 change +0.0081, 95% CI crossing zero) despite near-perfect detection recall, a grounding-corruption fault is significantly recovered (+0.0099), and a system-level reranker fault is never detected at all (recall 0.000) because the selected verifier’s fitted weights place essentially no mass on reranker-derived features — the same structural gap RQ1 identifies, now visible in a concrete failure mode. We report a real implementation defect found mid-study and its correction as part of the record, since fixing it materially changed prior results. Taken together, these findings map where observability-guided control earns its keep and where it does not, under a protocol designed so that map is trustworthy.

Index Terms—query expansion, information retrieval, agentic systems, AI observability, retrieval-augmented generation, large language models, site reliability engineering

I. INTRODUCTION

Large language models can generate useful pseudo-documents, alternative phrasings, and disambiguating context for a search query [1], [2], and corpus-grounded generation — conditioning the rewrite on retrieved evidence rather than the query text alone — improves on naive query-only rewriting. Existing work has established that LLM query expansion *can* help. It has not established when an operational system should trust an expansion enough to use it. In practice, expansion is applied unconditionally to every query or not at all; neither policy is safe, since unconditional rewriting is reliably net-harmful on a measurable fraction of queries, particularly underspecified or already-well-served ones.

We treat this as a control problem rather than a generation problem. Borrowing the observe-diagnose-act-recover discipline of site reliability engineering [3] and the trace/metric vocabulary of OpenTelemetry [4], [5], we build an agent whose action space is a small set of interpretable, corpus-grounded expansion operators, whose control loop alternates actions with structured observations in the manner of ReAct [6], and whose accept, roll-back, replan, and abstain decisions are driven by runtime telemetry rather than the language model’s own self-assessment. The central question is not whether an LLM can produce a useful expansion — that is established — but whether *runtime observability* can make expansion selective, recoverable, and cost-aware. This paper reports what we found rigorously answering that question, including where the answer was no, as a necessary complement to the mostly-positive existing literature.

A. Research Questions

RQ1 (Detection). Can qrel-free retrieval and agent-runtime telemetry identify unhealthy retrieval states and predict whether query expansion is likely to help or harm, without access to relevance judgments at inference time?

RQ2 (Control). Does an agent that conditions expansion actions on that telemetry outperform an otherwise identical *static*, failure-conditioned expansion pipeline — the same retriever, grounding mechanism, generator, and fusion method, differing only in whether a closed control loop can observe an action’s consequences and recover from a bad one?

RQ3 (Recovery). After a harmful rewrite or a simulated system-level fault, can observability-driven verification detect the degradation and select an appropriate recovery action to restore retrieval quality, under a bounded action budget — and does this hold even for a

verifier whose general-purpose detection power (RQ1) is only weakly informative?

B. Contributions

We contribute (1) an agent architecture treating LLM query expansion as a bounded, typed-tool action inside a closed observability loop, with a telemetry model and failure taxonomy behind it; (2) a frozen train/validation/test protocol (Section IV-B) under which every retrieval-config, verifier-selection, and threshold decision is made on held-out validation before one single-touch test evaluation, so RQ1–RQ3 report generalization results, not in-sample characterizations; (3) a corrected BM25 baseline, selected entirely on validation, that moves our TripClick TAIL retriever from below to above the published reference point and changes every downstream conclusion; (4) a full-scale, frozen-test evaluation of the closed-loop agent against a matched static comparator, plus a retrospective oracle analysis separating “no control opportunity exists” from “the opportunity exists but our detector cannot exploit it”; (5) a fault-injection recovery study showing which failure types a qrel-free verifier can and cannot recover from, tied back to which telemetry features it actually weights; and (6) an honest account of a real implementation defect, discovered and corrected mid-study.

II. RELATED WORK

LLM query expansion. Query2doc [1] generates pseudo-documents from an LLM and appends them to the query. MuGI [2] generates and fuses multiple expansions. Both treat expansion as an unconditional, one-shot generation step; neither conditions expansion on retrieval-state telemetry nor recovers from a harmful rewrite once issued.

Agentic control loops. ReAct [6] interleaves reasoning with actions and observations, the template for our observe-act-observe structure, but at a much smaller, more measurable action space: a fixed set of typed retrieval operators rather than free-form tool calls, keeping the policy interpretable and letting a lightweight, non-LLM controller drive it.

Observability and reliability engineering. OpenTelemetry’s trace/metric/log separation and GenAI semantic conventions [4], [5] supply a naming discipline for our telemetry. Google’s SRE framing of a Service Level Objective as a target range, not a single metric to optimize [3], motivates our Agent Search SLO: maximize quality subject to harm, cost, and latency constraints frozen *before* evaluation.

Benchmarks. TripClick [7] has genuine head/torso/tail query-frequency structure, unlike benchmarks that treat “hard” as a synonym for low-BM25-score. MS MARCO Chameleons [8] isolates queries resistant to reformulation; TREC DL 2019 [9] supplies deep NIST judgments; BEIR’s NFCorpus [10] is used here only for engineering iteration.

III. SYSTEM DESIGN

A. Agent State and Control Loop

The agent’s state at step t is $s_t = (q_0, q_t, R_t, O_t, H_t, B_t)$, where q_0 is the immutable original query, q_t the active search expression, R_t the current ranking, O_t the observability state, H_t the episode’s action/observation history, and B_t the remaining action budget. The control loop is

OBSERVE $\rightarrow$ DIAGNOSE $\rightarrow$ ACT $\rightarrow$ RETRIEVE
 $\rightarrow$ OBSERVE $\rightarrow$ VERIFY
 $\rightarrow$ {ACCEPT, ROLLBACK, REPLAN, STOP}.

The critical property this loop is built to test is that the second decision (accept/rollback/replan/stop) depends on evidence the agent’s own first action created — as opposed to a static pipeline that receives the same query, initial ranking, evidence, generator, and verifier, but executes exactly one predetermined route. Isolating this

dependency, not simply comparing against unmodified BM25, is the paper’s central experimental contrast (Section VI).

The action space is deliberately small and interpretable:

$A = \{\text{NOEXPAND, VOCABULARY, CONTEXT, AMBIGUITY, ENTITYNORMALIZE, ROLLBACK, REPLAN, STOP}\}.$

VOCABULARY maps informal user language onto corpus-compatible terminology; CONTEXT adds a corpus-supported missing facet to an underspecified query; AMBIGUITY disambiguates toward the interpretation the evidence best supports; ENTITYNORMALIZE resolves aliases, abbreviations, and canonical-name mismatches. The LLM is invoked through exactly one typed tool, `generate_expansion(action, query, evidence)`; it is never the sole arbiter of whether its own output is used. That judgment is made by a separate, non-generative controller (Section III-D).

B. Observability Model

Every query episode produces one trace with spans `retrieve_original` $\rightarrow$ `build_evidence` $\rightarrow$ `plan_action` $\rightarrow$ `generate_expansion` $\rightarrow$ `retrieve_expanded` $\rightarrow$ `rerank` $\rightarrow$ `verify` $\rightarrow$ `recover_or_accept` $\rightarrow$ `fuse`. The observation vector O_t spans three telemetry families. *Retrieval telemetry*: top score, score margin, normalized score entropy, query-term coverage, mean rare-term IDF, entity overlap, top- k ranking overlap against the previous state, reranker/BM25 disagreement, lexical result coherence, and ranking stability under a small controlled query perturbation. *Agent telemetry*: selected operator, trajectory length, previous verifier score, rewrite-to-original semantic drift, introduced entities, repeated actions, and remaining budget. *System telemetry*: per-tool latency, token counts, and tool errors or timeouts. Every feature here is computable without relevance judgments, which is what makes the resulting verifier usable at inference time.

C. Failure Taxonomy

We label episodes into ten failure categories (vocabulary mismatch, underspecification, ambiguity, entity mismatch, intent drift, confirmation bias, retriever disagreement, tool degradation, budget exhaustion, healthy retrieval), each with an observable telemetry signature and preferred recovery action, giving the observability layer something concrete to predict rather than treating it as post hoc dashboards; this directly motivates Section VII’s fault-injection protocol.

D. Verifier and Controller

The verifier, `verify(previous_state, new_state)`, produces a scalar judgment of whether an action’s consequences look like an improvement, using telemetry alone. Rather than hand-picking its weights, we calibrate them against click-derived qrels (Section IV): for a sample of episodes, we execute one expansion action, compute the true nDCG@10 change of the resulting ranking against those labels, and fit the verifier’s weights to predict that change from the telemetry features described in Section III-B. Click labels are available only during this offline calibration; the deployed verifier never sees one. We treat this fitted model as an artifact — trained by a dedicated script and loaded at inference time — rather than a set of constants transcribed from a one-off calibration run, so that recalibration is a script re-run, not a source edit. The recovery controller is a small deterministic finite-state machine: below a calibrated harmful threshold the agent rolls back to the pre-action ranking; above a calibrated accept threshold it fuses the new ranking in via reciprocal-rank fusion; in between it replans with an untried operator, subject to a hard cap of two model calls per query. Both thresholds are calibrated empirically (Section IV), not guessed.

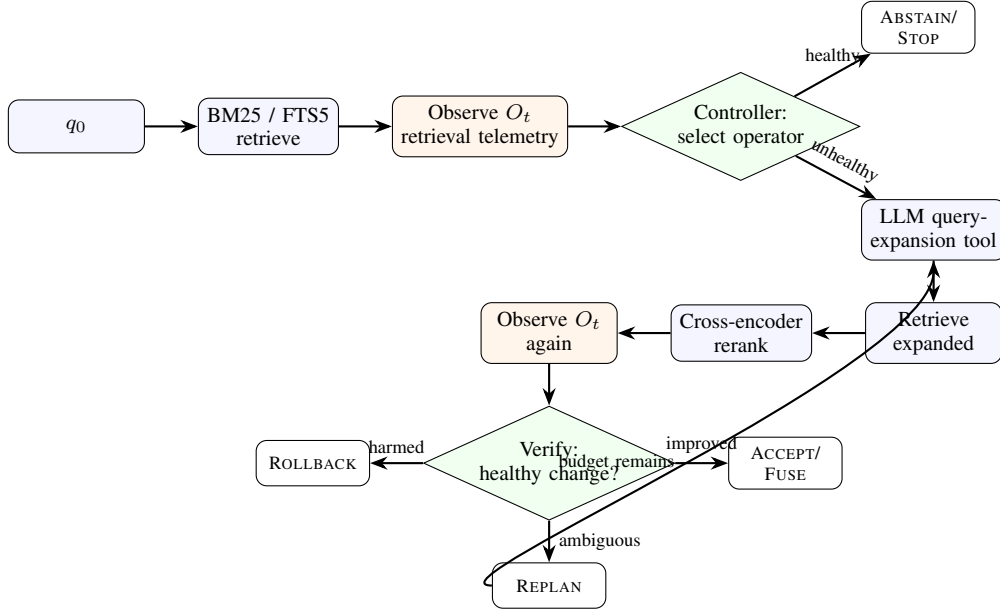


Fig. 1. The observability-guided control loop. Telemetry O_t (orange) is computed after every retrieval, not only episode-end, and drives both operator selection and the accept/rollback/replan/stop decision. The static comparator (Section VI) runs the same chain once, with no path back from VERIFY to the LLM tool.

IV. EXPERIMENTAL SETUP

A. Datasets

TripClick [7] is the primary benchmark: 1,523,878 documents, split evenly across head/torso/tail frequency buckets, each further split into train/validation/test (1,175 queries per split per bucket) — the only candidate dataset reflecting genuine observed query-frequency structure rather than treating “tail” as a synonym for “hard for BM25.” All results concern the TAIL split. Labels are click-derived, not human-assessed (binary raw clicks, matching the label type this split’s published reference point uses [7]); we use “qrel” and “click label” interchangeably, neither implying editorial judgment. NFCorpus [10] (3,633 documents, 323 queries) was used only for engineering iteration.

B. Experimental Protocol

A first pass at this study calibrated the verifier and thresholds on the same episodes later used to report RQ1–RQ3 — an in-sample characterization a reviewer could correctly read as overstating generalization. We froze a stricter protocol instead, in a committed config rather than left implicit: TAIL *validation* may be used for any retrieval-config, verifier, or threshold decision; TAIL *test* may be touched at most once per research question, purely for evaluation, after every upstream decision is frozen, with no decision afterward conditioned on a test-set outcome. This discipline directly produced the paper’s first finding, below.

C. A Corrected Retrieval Baseline

BM25 over an unstemmed FTS5 index scored nDCG@10 0.228 on TAIL test — 15% below the 0.267 reference reported for this exact split and label type [11]. Selecting entirely on validation (never test), we found two config choices that close this gap: Porter stemming (val nDCG@10 0.235 → 0.270) and weighting the title field ten times body weight in a title/body-split FTS5 index (val nDCG@10 climbing to a clear plateau at that ratio, 0.327, MRR peaking there and declining beyond it). A single, one-time sanity check of the frozen config against the withheld test set — not a selection criterion — gives nDCG@10 0.315, *above* the published reference. We report this because an under-tuned baseline mechanically inflates the apparent

value of anything compared against it: every downstream conclusion in this paper (Sections VI–VII) changed once this fix was in place, and we regard resolving it as a precondition for the rest of the study rather than an implementation footnote.

D. Implementation

Sparse retrieval uses BM25 over a SQLite FTS5 index [12] rather than a JVM Lucene index, forced by a JVM crash under Pyserini [13] on our target hardware; two further lessons surfaced at this corpus’s scale, both silent-corruption risks in the same category as Section IV-C’s baseline gap and Section VII-A’s telemetry defect: FTS5’s default MATCH combining operator is *conjunctive* (silently required every multi-term query’s terms to co-occur, collapsing recall without an error), and FTS5 indexes only the MATCH column (a companion lookup, e.g. document identifier, degrades to a full table scan — invisible at small scale, catastrophic at 1.5M rows, an 18× slowdown until a companion indexed table was added). Cross-encoder reranking (cross-encoder/ms-marco-MiniLM-L6-v2 [14]) is applied to the top 100 BM25 candidates, reranking the top 50. Generation uses a locally hosted, 4-bit quantized 2–3B-parameter model with a hard cap of two calls per query episode and a 32–64 token structured output, so the language model is never the entire decision surface, only one typed tool within it.

E. Verifier Calibration Methodology

Following Section IV-B, we collect calibration episodes (one expansion action per query, true nDCG@10 change against click labels) on TAIL *validation* ($n = 1147$ –1168 depending on feature family), and compare three feature families against two model classes each, to characterize the detection ceiling rather than accept the first plausible fit: (i) six retrieval-telemetry deltas; (ii) the same six plus four reranker- and length-derived features; and (iii) family (ii) plus three dense-embedding features. We compare a fixed-weight baseline, a 5-fold cross-validated linear fit, and a 5-fold cross-validated GBDT fit, on each family, against true nDCG@10 change under Pearson correlation (Table I) — this selects a model, and is never itself reported as evidence the signal generalizes (Section V reports that separately, from one frozen touch of test). Recovery thresholds are

TABLE I

MODEL SEARCH ON TAIL *validation* ($n = 1147$ – 1168): PEARSON CORRELATION VS. TRUE NDCG@10 CHANGE. SELECTS A MODEL; NOT ITSELF THE GENERALIZATION RESULT (THAT IS TABLE II).

Model, feature family	Val Pearson r
v0 hand-picked weights (6 features)	+0.020 (n.s., $p=0.50$)
Linear, 10 features (lexical+reranker)	−0.014
GBDT, 10 features	+0.001
Linear, 13 features (+dense embedding)	−0.015

TABLE II

RQ1’S FROZEN RESULT: THE SELECTED VERIFIER EVALUATED *once* ON WITHHELD TAIL TEST ($n = 1157$), NEVER TOUCHED DURING SELECTION.

Metric	Test value
Pearson r vs. true nDCG@10 change	0.114 ($p < 0.001$)
ROC-AUC (detecting harm)	0.552
Average precision (prevalence 0.330)	0.364

calibrated the same way, by checking which candidate score cutoffs on validation actually separate true-harmed episodes from the rest.

F. Metrics

Retrieval quality is nDCG@10, Recall@100, and MRR, plus the fraction of queries helped/unchanged/harmed relative to a baseline. For the recovery study (Section VII) we report Detection Recall $P(\text{intervened} \mid \text{harmful})$, False Alarm Rate $P(\text{intervened} \mid \text{healthy})$, and recovery gain in mean nDCG@10 with vs. without recovery on identically corrupted episodes. We additionally report a retrospective *oracle ceiling*: per query, $\max(\text{baseline}, \text{treatment})$ under true click labels — the outcome a perfect, non-deployable, qrel-using selective policy would achieve, diagnosing whether headroom exists independent of whether our qrel-free verifier can find it (Section VI). Significance is via paired bootstrap (10,000 resamples) throughout.

V. RQ1: DETECTION

Nothing clears noise on validation: every candidate — a literal hand-picked baseline, linear and gradient-boosted fits on the deployed 10-feature set, and a fit augmenting it with three dense-embedding features — lands at or below $|r| = 0.02$. We select the 10-feature linear model anyway, as the only candidate actually fit on the corrected pipeline’s feature distribution rather than a stale or untested one, and deploy it *despite* its own weak validation estimate, per Section IV-B: no decision here is conditioned on how well it does on test.

The frozen model generalizes better than its noisy validation estimate suggested: $r = 0.114$ on test is small but real ($p < 0.001$, $R^2 \approx 1.3\%$), and both ROC-AUC and average precision sit modestly above their no-skill baselines (Figure 2). We read the val/test gap honestly rather than as vindication: cross-validation on $\sim 1,150$ episodes is a noisy estimator for an effect this small, and a single larger test set landing positive does not retroactively make the selection procedure sound — it means this particular frozen artifact happens to generalize, which we could not have known before freezing it. **RQ1’s answer:** qrel-free telemetry carries a real but weak detection signal that survives out-of-sample evaluation, but that signal was never distinguishable from noise on the data used to select it.

VI. RQ2: CONTROL

Once BM25 itself is corrected (Section IV-C), the picture changes entirely from a preliminary pass on the under-tuned baseline: neither Static QE nor Agent significantly outperforms

TABLE III

RQ2, FROZEN RESULT: FULL TRIPCLICK TAIL TEST SET ($n = 1175$), CORRECTED RETRIEVAL BASELINE (SECTION IV-C). STATIC QE AND AGENT SHARE RETRIEVER, GROUNDING, GENERATOR, VERIFIER, AND FUSION; AGENT ADDITIONALLY HAS A CLOSED OBSERVE-VERIFY-RECOVER LOOP.

System	nDCG@10	Recall@100	MRR
BM25 only	0.3152	0.7750	0.3014
Static QE	0.3159	0.7759	0.2993
Agent	0.3161	0.7770	0.3015

TABLE IV

ORACLE SELECTIVITY CEILING, SAME $n = 1175$ EPISODES AS TABLE III. ORACLE PICKS $\max(\text{BM25}, \text{TREATMENT})$ PER QUERY UNDER TRUE CLICK LABELS.

Comparison	nDCG@10 gain	95% CI
Oracle-static vs. BM25	+0.0246	[0.0197, 0.0296]*
Oracle-static vs. Static QE	+0.0239	[0.0191, 0.0292]*
Oracle-agent vs. Agent	+0.0276	[0.0226, 0.0332]*

*: significant, $p < 0.001$.

BM25 alone (static: mean nDCG@10 difference +0.0007, 95% CI [−0.0066, 0.0079]; agent: +0.0009, 95% CI [−0.0069, 0.0087]), and Agent does not significantly outperform Static QE (+0.0002, 95% CI [−0.0027, 0.0032]; changes the outcome on 3.9% of queries, 2.1% helped and 1.7% harmed). The agent still costs 24% more model calls (1.22 vs. 0.98 per query) and 36% higher mean latency (15.1 s vs. 11.1 s, excluding episodes inflated by machine-sleep intervals during this run) for no offsetting gain. **On natural queries, neither expansion flavor moves the needle once the retriever is competitive** — materially different from what a weak baseline would have supported.

A. Is There Any Headroom at All?

A null result here is ambiguous: either no meaningful control opportunity exists on this corpus, or one exists but this verifier cannot find it. We resolve this with a retrospective oracle (Section IV): per query, take whichever of BM25 or the treatment ranking true click labels say is better. This oracle is not deployable — it uses labels unavailable at inference time — and exists purely to bound the second explanation.

The answer is unambiguous: substantial headroom exists. A perfect selective controller would gain +0.0246 nDCG@10 over BM25 — roughly $35\times$ what the deployed verifier’s Static QE actually captures (+0.0007) — and +0.0239 over what static expansion achieves in practice, both significant at $p < 0.001$. This is not a corpus with nothing to find: on this same population, static expansion helps 11.6% of queries and harms 10.6% relative to BM25 (Table III’s underlying per-query comparison) — a real, roughly-balanced mix of genuine gains and genuine harms that a perfect selector could exploit and a blind accept-everything policy cannot. The failure is squarely in *detection*: RQ1’s verifier, at $r = 0.114$ on frozen test, is far too weak to discriminate the helped 11.6% from the harmed 10.6% well enough to move aggregate quality. This is the paper’s clearest single diagnostic — it converts RQ2’s null result from “maybe there’s nothing here” into “there is something here, and our qrel-free detector cannot see it.”

Table V shows a controller that intervenes often (38.9% ROLL-BACK or REPLAN) without those interventions paying off in aggregate: the frozen verifier’s thresholds (Section IV-E) are wide, since nothing on validation cleanly separated harmful from healthy episodes, so the controller second-guesses frequently but not accurately. **RQ2’s answer is no**, with the oracle analysis clarifying *why*: an observability-guided closed loop built on this verifier does

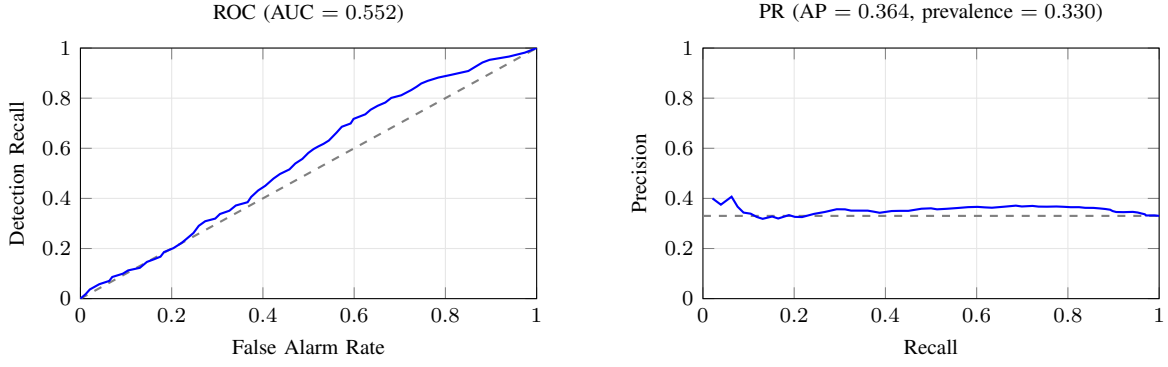


Fig. 2. RQ1’s frozen result: ROC and precision-recall curves for the selected verifier’s score against true harm (nDCG@10 $\Delta < 0$), swept across thresholds on the withheld TAIL *test* set ($n = 1157$, single touch). The curve sits consistently, but only modestly, above the no-skill baseline in both panels (dashed) — Table II as an operating-point trade-off rather than a single summary statistic.

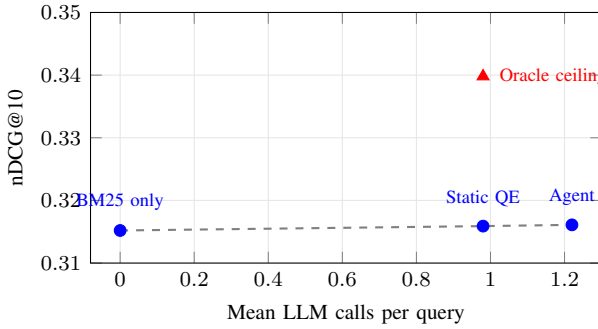


Fig. 3. Retrieval quality vs. LLM-call budget, full TripClick TAIL test set. BM25, Static QE, and Agent are effectively co-located; the oracle ceiling (Table IV) at the same call budget as Static QE shows the achievable quality is far above what either deployed system reaches.

TABLE V
AGENT CONTROLLER-DECISION FREQUENCY, FULL TRIPCLICK TAIL TEST SET ($n = 1175$).

Decision	Count	%
ACCEPT	699	59.5
ROLLBACK	331	28.2
REPLAN	127	10.8
ABSTAIN (no expansion)	18	1.5

not outperform a matched static pipeline, not because no control opportunity exists, but because this verifier cannot exploit it.

VII. RQ3: WHICH FAILURES ARE RECOVERABLE?

RQ1 and RQ2 leave open whether observability has *any* practical value, or whether a weak general detector is simply useless. We test a narrower claim: that the same weak verifier can still serve as a safety net against *large, obvious* failures even though it cannot reliably judge *subtle, natural* ones. We inject three fault types at their natural point of corruption — an unsupported entity appended to the expansion, a grounding passage swapped for a non-supporting one, and the reranker disabled — and compare, on identical corrupted episodes, an always-accept ablation against the real calibrated controller.

A. A Defect That Changed the Answer

An early pass at this study found the reranker-disabled fault essentially undetectable (recall 0.185) while the other two looked

TABLE VI
RQ3, FROZEN RESULT: FAULT INJECTION RECOVERY, CORRECTED PIPELINE, FULL TRIPCLICK TAIL TEST SET ($n = 1157$; 18 OF 1175 EPISODES SKIPPED FOR NO EXPANSION). GAIN IS MEAN NDCG@10 WITH MINUS WITHOUT OBSERVABILITY-DRIVEN RECOVERY ON IDENTICALLY CORRUPTED EPISODES; CI FROM 10,000-RESAMPLE PAIRED BOOTSTRAP.

Fault type	Recall	False Alarm	Gain (95% CI)
Entity injection	0.974	0.939	+0.0081 [−0.0029, +0.0189] NS
Grounding corruption	0.528	0.469	+0.0099 [+0.0016, +0.0182]*
Reranker disabled	0.000	0.002	+0.0000 NS

NS: not significant, CI straddles zero. *: significant, positive.

strong. Tracing this revealed a real defect: the reranked ranking was computed, at real latency cost, and fed into telemetry, but never actually applied to the ranking the agent accepted or evaluated — disabling the reranker was mechanically a no-op regardless of verifier quality. Fixing it exposed a second, subtler defect it was entangled with: the verifier’s telemetry was still computed from the pre-reranker ranking, judging a different top- k than the one being accepted. We corrected both — the reranked top- k now leads the accepted ranking, and telemetry is reordered to match it before scoring — and re-ran calibration to confirm the deployed verifier and its thresholds did not need retraining on the corrected telemetry (corrected-pipeline Pearson $r = 0.146$ vs. the original 0.134; a fresh refit did not clear the bar to replace it). We report this defect in full, not as a footnote, because a component computed purely for observability is architecturally easy to leave disconnected from the outcome it explains, and that disconnection produces results that look coherent — in this case, unrealistically strong — until someone investigates why an otherwise well-motivated intervention shows no effect.

B. Frozen Results

The three fault types split into three distinct outcomes, each with a different explanation, so “is recovery useful” has no single answer — only a per-failure-type one. **Entity injection** is detected almost every time (recall 0.974) but flagged almost indiscriminately (false alarm 0.939): the controller treats nearly any entity-injected rewrite as suspect whether or not the injection was actually benign, and rolling back that many healthy episodes offsets the genuine catches, leaving a positive but not significant net effect. **Grounding corruption** is the one clean recovery success (+0.0099, $p < 0.05$): moderate recall (0.528) paired with a comparable false alarm rate (0.469) is discriminating enough, on this fault type specifically, to produce a real net gain. **Reranker disabled** is never detected at all (recall 0.000): the selected verifier’s fitted weights (Section IV-E) place essentially zero mass

on reranker-derived features (`reranker_top_score` +0.0007, `reranker_score_margin` +0.0007, against `coverage_gain` +0.2051 and `disagreement` -0.2528), so a fault that manipulates reranker behavior specifically is structurally invisible to it — the same feature-weighting profile selected purely from RQ1’s validation-set model search, now surfacing as a concrete blind spot rather than an abstract correlation ceiling.

RQ3’s answer is that recoverability is fault-specific, not a property of the verifier in general: content-level corruption is recoverable when the verifier’s telemetry is discriminating for that specific corruption pattern (grounding), a wash when it is not (entity injection, blunt detection swamped by false alarms), and a system-level fault is entirely unrecoverable when it manipulates a component the verifier’s fitted weights do not track at all (reranker). This traces directly back to RQ1: a verifier is not uniformly weak or strong across failure types, it is weak or strong *per feature the failure happens to perturb*.

VIII. DISCUSSION

Read together under the frozen protocol, RQ1–RQ3 and the oracle analysis sketch one coherent finding rather than four independent ones. RQ1’s detection signal is weak but real out-of-sample ($r = 0.114$ on test, despite a validation estimate indistinguishable from noise): not zero, not strong. RQ2 shows that a closed control loop built on that signal adds nothing over a matched static pipeline on a properly-tuned retrieval baseline. The oracle analysis (Section VI-A) is what turns those two facts into a diagnosis rather than a shrug: real, large control headroom exists (+0.0246 nDCG@10 available to a perfect selector) and the deployed verifier captures almost none of it, so the bottleneck is *specifically* detection quality, not an absent opportunity or a control-loop design flaw. RQ3 sharpens this further by showing detection quality is not one number but a per-feature property: the verifier recovers a fault type when its fitted weights happen to track the features that fault perturbs (grounding corruption), and is structurally blind to a fault type when they do not (reranker disabled, weights near zero on every reranker-derived feature). A weak overall correlation is therefore not just an aggregate ceiling; it decomposes into specific, predictable blind spots that a practitioner could in principle inspect *before* deployment by asking which features a candidate fault would perturb and whether the fitted verifier weights those features at all.

Two things here were found only by refusing a convenient result: the BM25 baseline (Section IV-C) looked adequate until checked against a published reference, reversing RQ2’s conclusion entirely; the reranker-wiring defect (Section VII-A) looked like a strong recovery result until one fault type’s implausible near-zero effect prompted investigation. Both are cases where a component — a retrieval baseline, an observability signal — is architecturally easy to leave unchecked against an external reference, and the resulting error looks coherent rather than broken. A frozen protocol with a mandatory external sanity check (Section IV-B) structurally defends against the first; the second we only found by distrusting a result that seemed too good, which no protocol automates.

IX. LIMITATIONS

The oracle ceiling (Section VI-A) is retrospective and uses test-set click labels directly — a valid upper-bound diagnostic, never a deployable method. The generator is a locally hosted, quantized 2–3B parameter model under a strict two-call budget; we do not know whether a larger or API-hosted generator would change RQ1’s ceiling or RQ2’s null result. All results concern TripClick’s TAIL split; HEAD, TORSO, MS MARCO Chameleons, and TREC DL 2019/2020 are left to future work. Click-derived qrels are an imperfect relevance proxy inherited from the benchmark, not introduced by our protocol. The corrected baseline’s title-weight ratio (10× body) was selected on a genuine validation-set plateau, not cherry-picked,

but we have not tested whether it transfers beyond TripClick’s title-informative health-literature abstracts. RQ3’s “per-feature blind spot” explanation is a plausible mechanism consistent with the observed pattern, not one we verified with faults targeting other feature subsets.

X. CONCLUSION

We evaluated a closed-loop, observability-guided query-expansion agent under a train/validation/test discipline strict enough to survive the scrutiny that sank an earlier, leakier pass at this same study — which itself surfaced the paper’s first finding: an honestly-selected BM25 baseline fix inverted every downstream conclusion. On the corrected baseline, the agent does not outperform static expansion, and neither outperforms BM25; but the oracle analysis shows this is a detection failure, not an absent opportunity, and the recovery study shows that failure is a per-feature property, not a uniform ceiling. We report both defects found by refusing a convenient result as part of this record, in the belief that the discipline required to catch them is worth more to this line of research than a single flattering number would have been.

ACKNOWLEDGMENT

The authors thank the TripClick maintainers for access to the dataset under its non-commercial research license.

REFERENCES

- [1] L. Wang, N. Yang, and F. Wei, “Query2doc: Query expansion with large language models,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. [Online]. Available: <https://aclanthology.org/2023.emnlp-main.585/>
- [2] L. Zhang, Y. Wu, Q. Yang, J. Lin *et al.*, “Exploring the best practices of query expansion with large language models,” in *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024. [Online]. Available: <https://aclanthology.org/2024.findings-emnlp.103/>
- [3] Google, “The site reliability workbook: Defining SLO,” 2026. [Online]. Available: <https://sre.google/sre-book/service-level-objectives/>
- [4] OpenTelemetry Authors, “OpenTelemetry signals,” 2026. [Online]. Available: <https://opentelemetry.io/docs/concepts/signals/>
- [5] —, “OpenTelemetry GenAI semantic conventions 1.44.0,” 2026. [Online]. Available: <https://opentelemetry.io/docs/specs/semconv/>
- [6] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “React: Synergizing reasoning and acting in language models,” *arXiv preprint arXiv:2210.03629*, 2022. [Online]. Available: <https://arxiv.org/abs/2210.03629>
- [7] N. Rekabsaz, O. Lesota, M. Schedl, J. Brassey, and C. Eickhoff, “TripClick: The log files of a large health web search engine,” 2021. [Online]. Available: <https://tripdatabase.github.io/tripclick/>
- [8] Microsoft Research, “MS MARCO chameleons: Challenging the MS MARCO leaderboard with extremely obstinate queries,” 2026. [Online]. Available: <https://www.microsoft.com/en-us/research/publication/ms-marco-chameleons-challenging-the-ms-marco-leaderboard-with-extremely-obstinate-queries/>
- [9] NIST, “Text REtrieval conference (TREC) 2019 deep learning track,” 2019. [Online]. Available: <https://trec.nist.gov/data/deep2019.html>
- [10] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych, “BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models,” in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021. [Online]. Available: <https://github.com/beir-cellar/beir>
- [11] N. Rekabsaz, O. Lesota, M. Schedl, J. Brassey, and C. Eickhoff, “Establishing strong baselines for TripClick health retrieval,” in *European Conference on Information Retrieval*, 2022. [Online]. Available: <https://arxiv.org/abs/2201.00365>
- [12] SQLite Consortium, “SQLite FTS5 extension,” 2026. [Online]. Available: <https://www.sqlite.org/fts5.html>
- [13] J. Lin, X. Ma, S.-C. Lin, J.-H. Yang, R. Pradeep, and R. Nogueira, “Pyserini: Reproducible IR research with sparse and dense representations,” 2026. [Online]. Available: <https://github.com/castorini/pyserini>
- [14] Hugging Face, “cross-encoder/ms-marco-MiniLM-L6-v2,” 2026. [Online]. Available: <https://huggingface.co/cross-encoder/ms-marco-MiniLM-L6-v2>